

文章编号: 1007-130X(2003)04-0094-05

以基本块为单位的非顺序指令预取^{*}

The Non-Sequential Instruction Prefetching Based on Basic Blocks

沈立, 戴葵, 王志英

SHEN Li, DAI Kui, WANG Zhi-ying

(国防科技大学计算机学院, 湖南长沙 410073)

(School of Computer Science, National University of Defense Technology, Changsha 410073, China)

摘 要 取指令能力的高低对微处理器的性能有很大影响。指令预取技术能够有效地降低指令 Cache 的访问失效率, 提高微处理器的取指令能力, 进而提高微处理器的性能。本文提出了一种由分支指令指导的、以基本块为单位的非顺序指令预取技术, 每次预取将一个完整的基本块读入指令 Cache。这种方法使用静态策略分析程序行为, 实现所需的硬件复杂度低。模拟结果显示, 该方法能够有效地提高指令 Cache 访问的命中率。

Abstract Instruction supply can influence processor performance greatly. Instruction prefetching is an effective mechanism to reduce the instruction cache miss rate. This paper proposes a non-sequential instruction prefetching mechanism, which is directed by branch instructions and prefetches a whole basic block each time. Experimental results show it can increase the instruction cache hit rate effectively. In addition, it predicts program behaviors statically so that it does not need any complex hardware to predict and restore.

关键词 基本块 指令 Cache 指令预取 分支预测

Key words basic block instruction cache instruction prefetching branch prediction

中图分类号 TP303

文献标识码 A

1 引言

现代高性能微处理器通常可以分为两个部分: 前端指令处理模块和执行模块。前端指令处理模块主要负责从存储器中读取指令并进行指令译码, 而执行模块主要负责指令的执行以及有效结果的保存。这两个模块通过指令联系在一起, 一个负责提供指令, 一个负责执行指令, 是典型的生产者与消费者的关系。能否充分发挥执行模块的效能, 取决于前端模块是否能够提供足够多的

指令供其执行, 多流出微处理器尤其如此。常用的解决方法是扩大基本块, 并在一个周期内读出整个基本块或多个基本块。高级编译技术有助于实现这一方案, 将若干个基本块组合成一条路径 (Trace)^[1] 或一个超块^[2] 可以扩大基本块, 或者沿着多条执行路径进行指令调度。

使用指令 Cache 可以缓解微处理器和存储器之间的性能差距, 提高微处理器的取指令能力。指令 Cache 访问失效严重影响了微处理器的性能提高。指令预取技术通过分析程序行为, 提前把即将被访问的指令从存储器中读入指令 Cache, 从

* 收稿日期: 2002-07-09; 修订日期: 2002-09-16

基金项目: 国家自然科学基金资助项目 (60173040, 6993303)

作者简介: 沈立 (1976-), 男, 陕西宝鸡人, 博士生, 研究方向为高级计算机体系结构及编译技术; 戴葵, 副教授, 研究方向为高级计算机体系结构、微处理器设计技术; 王志英, 教授, 博士生导师, 研究方向为高级计算机体系结构、微处理器设计。

通讯地址: 410073 湖南省长沙市国防科技大学计算机学院; Tel: (0731) 4575963; E-mail: shen2k@hotmail.com

Address: School of Computer Science, National University of Defense Technology, Changsha, Hunan 410073, P. R. China

而降低指令 Cache 的访问失效率。许多研究^[3,4]表明,指令预取能够有效地提高取指令部件的效率。本文将讨论一种以基本块为单位的非顺序指令预取技术。

研究指令预取技术主要应考虑以下三个问题:

(1) 由于分支指令的存在,运行时所执行的指令并非总是顺序的,如何确定被预取指令的地址?

(2) 无效预取包括两类:预取已经在 Cache 中的指令块;指令块被读入 Cache 后,在被替换出 Cache 之前没有被访问过。在这两种情况下,将指令块读入指令 Cache 毫无意义,反而浪费了取指令带宽。如何避免无效预取?

(3) 如果预取时间太早,可能会造成无效预取;如果预取时间太晚,又无法保证有充足的时间将指令读入指令 Cache。如何确定预取指令的流出时间?

衡量指令预取技术性能高低的标准主要包括:指令 Cache 失效率、无效预取比率以及能否尽可能早地进行预取(一般用预取完成率来衡量)。本文也将使用这三项指标对预取技术进行评价。

2 以基本块为单位的非顺序指令预取

只有正确地预测程序行为,才能实现精确的分支预取。分支指令的存在使得程序并非总是按照指令地址的顺序执行,不同的输入数据集又造成程序行为的不确定,因而只能采用一定的策略预测程序行为。预测程序行为的方法有三类:(1) 静态预测,利用程序以前运行时统计的 profile 信息在编译时进行预测;(2) 动态预测,利用硬件以及对程序运行的记录在程序执行时进行预测;(3) 动静态结合预测,利用 profile 信息进行编译优化,并在程序运行过程中进行动态预测。一般说来,静态预测准确率低,动态预测硬件开销大,第三种方法效果最好,但实现复杂度最高。但是,无论怎样预测,都不可能保证结果完全正确。当预测错误时,预取的指令(甚至包括某些已经被执行的指令)都是无效的,此时必须通过一定的机制恢复到预测前的状态,以确保程序执行的正确性。对于任何预测策略,错误恢复机制都是不可避免的。

2.1 常用的指令预取技术

最简单的指令预取技术为 Next-N-Line 预取。
万方数据

这是一种顺序预取技术,每次预取当前指令块后的 N 个块,通常在 $N = 4$ 或 8 时效果最好。其优点是实现简单,但效率不高,体现在无效预取指令多,并且不能对长距离跳转、过程调用、过程返回、间接跳转等分支指令进行有效处理。

Target-Line 预取使用一个预测表确定预取指令地址,表中记录了指令地址和对应的预取地址。每个指令块可能包含多条指令,分别对应预测表中某个项。这种技术可能无法保证有充足的时间完成预取,其效果的好坏受到预测表大小的影响,但其无效预取率低于 Next-N-Line 预取。

Wrong-Path 预取^[5]根据条件分支指令进行预取,但此次预取并不是为本次执行进行的,而是为了确保在下次执行该分支指令时,目标指令已经被读入指令 Cache。这种技术实现代价低,可能有足够的时间完成预取,但可能有较多的预取是无效的。

Markov 预取^[6]最初是为了实现数据预取提出的,每当访问 Cache 失效时进行预取操作。该机制维护一个预测表,表中记录了每个失效地址及其所对应的预取地址,失效地址和预取之间是一对多对应关系。采用这种方法,发出预取操作的时间可能足够早,但无法保证预取是否有效。

总的说来,这些技术能够在一定程度上降低指令 Cache 失效率,但主要问题在于:发出预取的时间不够准确,无法确保有足够的时间完成预取,同时也无法保证预取是否有效。本文提出了一种以基本块为单位的非顺序预取技术,该方法采用静态技术分析程序行为,在降低指令 Cache 失效率的同时,能够保证发出预取操作的时间足够早。另外,由于每次预取的指令根据基本块动态确定,因此,可以保证预取的有效性,降低无效预取率。

2.2 程序运行的流水线模型

基本块是程序中一个顺序执行的语句序列,其中只有一个入口和一个出口。对一个基本块而言,执行时只能从其入口进入,从其出口退出。对于一个给定的程序,可以把它划分为一系列的基本块。我们将一个基本块执行结束后下一个被执行的基本块称为它的后继基本块。在 RISC 机器中,一个基本块最多只有两个后继基本块。

根据 RISC 机器的这一特点,我们认为可以根据对分支指令的译码结果动态确定其后继基本块,从而消除流水线中的控制相关延迟。该流水线模型如图 1 所示,流水线中有两个取指令部件

负责同时读入当前基本块的两个后继基本块,当分支指令译码结束后,根据其译码结果选择某个指令部件中的指令送入译码器进行译码并执行,另一个取指令部件则开始读取当前基本块(原来的后继基本块之一)的后继基本块。

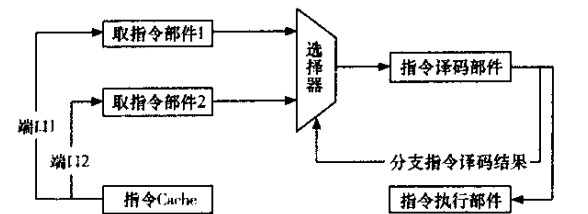


图1 程序运行的流水线模型

2.3 预取方法

根据 2.2 节介绍的流水线模型,可以以基本块为单位进行指令预取。预取指令的位置在基本块的第一条语句之后,这样就可以在一个基本块执行的同时,对其后继基本块进行预取。以基本块为单位进行预取,还可以在在一定程度上避免扩大预取指令的范围和不必要的预取。

为此,我们专门设计了一组指令完成预取操作,其语法格式如表 1 所示。为了处理方便起见,预取指令采用特殊编码,只需要根据一条指令的某几位即可判断它是否是预取指令,同时可以迅速读出预取地址。

表 1 预取指令语法格式

预取命令	描 述
fetch addr	预取从地址 addr 开始的基本块。
fetch addr1, addr2	分别预取从地址 addr1 和 addr2 开始的两个基本块。
fetch stack	主要处理过程返回语句,预取从栈顶单元地址开始的基本块。

预取流程为 程序开始运行时,前端模块将第一个基本块读入指令 buffer(如果基本块长度比较大,则每次读一部分),当预取指令被读入指令 buffer 后,说明已经开始执行基本块的第一条语句,与此同时应该执行预取指令,这样可以在执行当前基本块的同时预取它的后继基本块;一个基本块运行结束后,根据其运行结果读入并执行正确的后继基本块,并执行这个后继基本块中包含的预取指令,以此类推,直至程序运行结束。整个流程如图 2 所示,图中只包括与预取指令相关的内容。为了避免无效预取,如果指令块已经在 Cache 中,则不执行预取。

这里涉及到的一个关键问题是,如何判断一

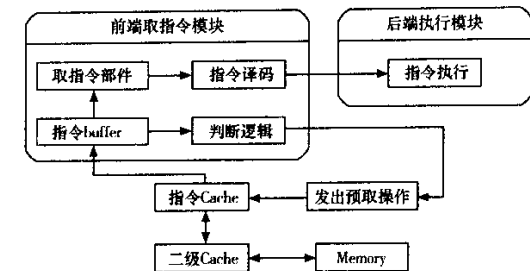


图2 以基本块为单位的预取技术程序运行流程图

个新的基本块已经被读入指令 buffer 并执行其中的预取指令。由于预取指令的编码很有特点,在指令 buffer 增加了一定的判断逻辑,负责判断指令 buffer 中的第一条指令是否是预取指令。如果是预取指令,则执行预取操作并将其从指令 buffer 中清除,否则将其送往指令译码部件进行译码。

2.4 预取指令定义

根据基本块的定义,基本块的出口语句都是能够改变程序计数器(PC)的值的语句,本文将其统称为分支指令。在 RISC 机器中,一个基本块可以有一个或两个后继基本块,这些后继基本块的地址都可以根据分支指令的类型确定。这样,根据不同类型的基本块出口语句(分支指令)就可以确定相应的预取基本块。

一般说来,分支指令可以分为四类:条件分支指令、直接无条件分支指令、间接无条件分支指令和过程返回语句。根据这四类指令的不同情况插入不同的预取指令。

(1) 条件分支指令。转移可能成功或失败,因此有两个后继基本块。在编译时可以获得转移成功和转移失败时的目标地址,同时预取当前基本块的两个后继基本块。预取指令为 fetch addr1, addr2。

(2) 直接无条件分支指令。转移总是成功,只有一个后继基本块。在编译时可以获得转移目标地址,预取目标基本块即可。预取指令为 fetch addr。

(3) 间接无条件分支指令。转移总是成功,只有一个后继基本块。转移目标地址保存在寄存器中,通常在编译时无法得到,因此,对于这类分支指令,我们不进行预取。

(4) 过程返回语句。转移总是成功,只有一个后继基本块。这类语句通常出现在过程调用返回时,由于过程调用可能出现嵌套,我们用一个栈来保存过程调用的返回地址(也就是预取地址),

每次从栈的顶端获得预取地址并完成预取。预取指令为 fetch stack。

之所以在执行一个基本块的第一条指令的同时,对其后继基本块进行预取,是因为基本块大小通常不会太大,这样做能够保证有充足的时间完成预取。

3 性能分析

除间接无条件转移指令外,被预取指令的地址在编译时即可确定,而且在编译时能够保证预取地址的准确性。

执行某个基本块第一条指令的同时,将对下一个将要被执行的基本块进行预取,这样可以保证有比较充足的时间完成预取。

采用这种机制,由于没有进行预测,只要指令被执行,就一定是有效的,因而第二种无效预取不存在。只有根据条件分支指令预取的指令才有可能无效的,指令在未被执行之前就被替换出指令 Cache 的情况,当且仅当指令位于条件分支指令未被执行的后继基本块中时才有可能发生。

3.1 模拟环境和测试程序

我们选择了以下的模拟环境 使用两级 Cache,指令 Cache 容量为 32K 字节,相联度为 4,块大小为 32 字节,二级 Cache 为数据指令混合 Cache,容量为 1M 字节 取指令部件的取指带宽为 32 字节/cycle,两级 Cache 之前的带宽为 8 字节/cycle ;Cache 的替换策略为 LRU,采用写回方式。取指令部件的指令缓冲器最多可以保存 16 条指令。

我们选择 SPECint95 程序中的程序作为测试程序,如表 2 所示。所有测试程序均用 GCC 编译,没有经过任何优化,编译的同时向程度中加入预取指令。

表 2 测试程序

测试程序	ICache 失效率	二级 Cache 失效率
Gcc	6.4%	0.012%
Perl	2.7%	0.009%
Vortex	10.8%	0.011%
Go	2.1%	0.010%
M88ksim	1.2%	0.021%
Li	1.9%	0.007%
Compress	3.2%	0.011%
Ljpeg	2.1%	0.015%

3.2 模拟结果及分析

表 3 列出了针对 SPECint95 测试程序的模拟万方数据

结果。从表中可以看出,使用以基本块为单位的预取技术,能够在保证无效预取率比较低的前提下,有效提高指令 Cache 的访问命中率。所有 SPECint95 测试程序的平均预取完成率为 97.025%,说明采用这种方法能够保证有足够多的时间完成预取。

表 3 模块测试结果

测试程序	ICache 失效率	预取完成率	无效预取率
Gcc	0.65%	97.2%	6.9%
Perl	0.99%	96.5%	7.7%
Vortex	0.56%	97.4%	10.1%
Go	0.35%	98.5%	4.3%
m88ksim	0.13%	96.7%	7.2%
Li	0.19%	98.3%	3.3%
Compress	0.23%	97.1%	6.1%
Ljpeg	1.01%	94.5%	9.3%

图 3 从指令 Cache 失效率、无效预取率和预取完成率三个方面比较了本文讨论到的所有指令预取技术的性能,其中 N4 代表 Next-N-Line 预取,T 代表 Target 预取,W 代表 Wrong-Path 预取,M 代表 Markov 预取,B 代表以基本块为单位的预取。Next-N-Line 预取 N = 4,Target 预取预测表大小为 32

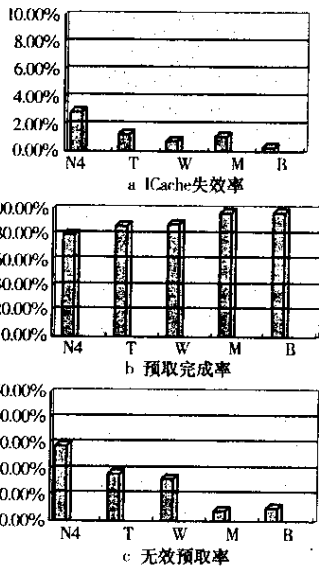


图 3 几种指令预取技术的性能比较

项。从图中可以看出,采用本文提出的预取方法,指令 Cache 失效率最低,预取完成率和无效预取率也比较理想。

4 结束语

本文提出了一种由分支指令指导的非顺序指令预取技术。该技术采用静态方法分析程序行为,因而可以避免使用复杂的分支预测和错误恢复硬件,降低了硬件复杂度。模拟结果显示,该方法能够有效地降低指令 Cache 的访问失效率、无效预取率,效果比较理想。

我们准备进行进行以下方面的研究：

(1) 如何对间接无条件分支指令进行预取。

(2) 目前,预取指令的判断和执行机制还不是很完善,需要进一步改进。

(3) 无论预取技术如何高明,只能将指令读入指令 Cache。能否为执行模块提供足够多的指令供其执行,还需要取指令机制的支持。也就是说,能否充分发挥指令 Cache 的性能,需要高效取指令机制的支持。

参考文献：

- [1] R Colwell, R Nix, J O'Donnell, et al. A VLIW Architecture for a Trace Scheduling Compiler[A]. Proc of the 2nd Int'l Conf on Architectural Support for Programming Languages and Operating Systems[C]. 1987. 180 – 192.
- [2] W Hwu, S Mahlke, W Chen, et al. The Superblock :An Effective Technique for VLIW and Superscalar Compilation[J]. The Journal of Supercomputing, 1993, 7 :229 – 248.
- [3] C Xia, J Torrellas. Instruction Prefetching of Systems Codes with Layout Optimized for Reduced Cache Misses[A]. 23rd Annual Int'l Symp on Computer Architecture[C]. 1996.
- [4] N Jouppi. Improving Direct-Mapped Cache Performance by the Addition of a Small Fully Associative Cache and Prefetch Buffers[A]. Proc of the 17th Annual Int'l Symp on Computer Architecture[C]. 1990.
- [5] J Pierce, T Mudge. Wrong-Path Instruction Prefetching [A]. 29th Int'l Symp on Microarchitecture[C]. 1996. 165 – 175.
- [6] D Joseph, D Grunwald. Prefetching Using Markov Predictors[A]. 24th Annual Int'l Symp on Computer Architecture[C]. 1990.